Robust OFDM Channel Modelling for Wi Fi CSI Sensing in GNU Radio

Reproduction, measurement and root-cause analysis of GNU Radio OFDM reliability issues #6891 and #7997, and a tested upstream contribution

Name	Enrollment no.
Hirva Vekariya	AU2440126
Tirth Pathar	AU2440238
Divyaraj Vaghela	AU2440251
Jahnavi Patel	AU2440024
Jinay Shah	AU2440296

Domain	Wi-Fi Sensing	Mode	GSoC / Open source (GNU Radio)
Primary issue	gnuradio#7997	Related issue	gnuradio#6891
Software	GNU Radio 3.10.12.0 (conda-forge), Python 3.11, Ubuntu 22.04		
Repository	github.com/divyarajv1-ship-it/g1_gsoc_wifisensing → M2_g1_gsoc_wifisensing/		

1. Summary

In M1 we selected two reliability problems in GNU Radio's OFDM chain: a flowgraph that freezes (#7997) and packets that are lost on a noise-free channel (#6891). In M2 we built a measurement harness, reproduced both behaviours, located their causes in the GNU Radio source, and prepared a tested contribution.

Main results

1. Packet loss (#6891) reproduced and quantified: with default settings, 4.1 % of back-to-back packets are lost on an *ideal* channel (500 packets × 5 seeds).

2. Cause narrowed down: the loss is identical, packet by packet, when TX and RX run in one flowgraph and when RX runs afterwards on the stored signal. So the scheduler is *not* the cause; the signal processing (Schmidl & Cox timing + Header/Payload Demux) is.

3. Fix validated: rolloff=4 in the transmitter, or one zero sample between bursts, gives 100 % delivery in every run, at every SNR ≥ 15 dB.

4. Counter-intuitive result: with the default TX, delivery gets *worse* as SNR increases (99.6 % at 15 dB → 94.4 % at 30 dB).

5. Freeze mechanism (#7997 symptom) demonstrated: if a packet does not fit a block's output buffer, that block's thread exits with Buffer too small for min_noutput_items , and the whole flowgraph stops without raising an error. We trigger it in two ways (large packets; a length tag leaking from TX to RX) and remove it in both.

6. Contribution: a new multi-packet regression test for qa_ofdm_txrx.py (upstream tests only single packets) and a one-line fix of the official ofdm_loopback.grc example. The test passes with the fix and fails without it.

2. Problem and its wireless-communication relevance

Wi-Fi sensing infers human activity from Channel State Information (CSI): the per-subcarrier channel estimate an OFDM receiver computes from each packet's preamble. Every received packet gives one CSI sample, so the CSI time series is only as regular as packet reception. A receiver that silently drops 4 % of packets in a *systematic* way (not at random) creates gaps in the CSI series; a flowgraph that freezes stops data collection entirely. Before GNU Radio can be used as a controlled CSI-generation testbed, these two failure modes must be understood.

ECE 310 concept	**Where it appears in M2**
OFDM, cyclic prefix, FFT	64-subcarrier OFDM, 16-sample CP, 80-sample symbols; each 50-byte packet = 12 symbols = 960 samples
Synchronization / detection	Schmidl & Cox timing metric and plateau detector decide where each packet starts
Noise, SNR, AWGN channel	SNR sweep 6–30 dB with GNU Radio's Channel Model
Modulation, detection, BER	BPSK header, BPSK payload (defaults); CRC32 rejects any packet with bit errors
Wireless network performance	Packet delivery ratio (PDR), lost packets, flowgraph stability

3. Methodology

3.1 Signal chain

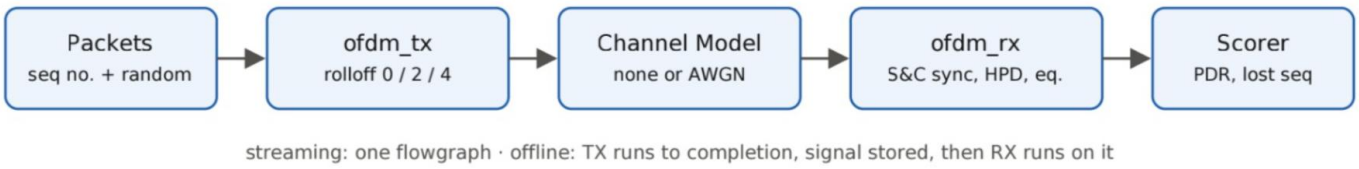


Figure 0. Measurement harness (Code/experiments/ofdm_packet_loss.py).

Design decisions, each made to remove a source of doubt:

• Sequence numbers. Bytes 0–1 of every packet hold its index, the rest are random. The issue thread shows that repeating payloads hide losses; with sequence numbers each lost packet is identified exactly.

• Two execution modes. *Streaming* runs TX→RX in one flowgraph, where the scheduler passes data in variable chunks. *Offline* stores the complete TX signal first. If losses differ, the scheduler is involved; if they are identical, it is not. This tests the open hypothesis in the #6891 thread that buffers or the scheduler cause the drops.

• Watchdog. Each trial runs in a child process with a timeout. A frozen flowgraph is recorded as HANG and the sweep continues.

• Fixed seeds (1–5) for payloads and channel noise, so every number in this report can be regenerated exactly.

• Flush packets. Three extra packets are sent after the measured ones and not scored, so the end of the stream cannot count as a loss.

• Tag isolation. As in the official example, the RX uses its own length-tag key (rx_len) and the Channel Model blocks tags (see §5.2 for why).

3.2 Metrics (defined before running the experiments)

Metric	Definition
Packet delivery ratio (PDR)	packets received with valid CRC32 and a correct sequence number ÷ packets sent
Lost packets	sequence numbers sent but never received
BER of delivered packets	bit errors in delivered packets ÷ bits checked (always 0 here, because ofdm_rx drops packets whose CRC fails)
Flowgraph stability	trial finishes before the timeout (OK) or not (HANG)
SNR	mean TX sample power (measured 50.98; the IFFT is not normalised) ÷ noise power noise_voltage²

3.3 Experiments

ID	Question	Varied	Size
E1	How large is the loss, does the scheduler matter, which fix works?	mode × {rolloff 0,1,2,4; gap 1,8}	60 trials × 500 packets
E2	Does packet length matter?	20, 50, 100, 200 bytes	20 trials × 500 packets
E3	How do loss and fix behave with noise?	SNR 6–30 dB × rolloff 0/4	130 trials × 300 packets
E4	When does the chain freeze?	packet size 50–800 B × default/ enlarged buffers	16 trials
E5	Do leaked length tags freeze the RX?	channel/no channel × block_tags × RX key	5 trials

4. Results

4.1 E1: packet loss and the fix (ideal channel)

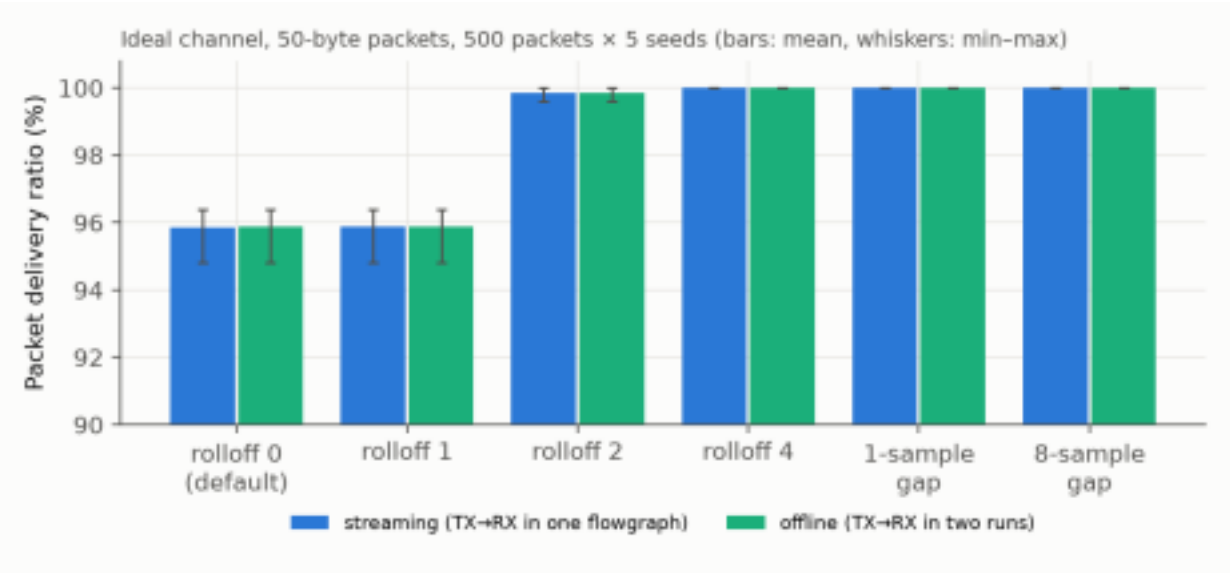


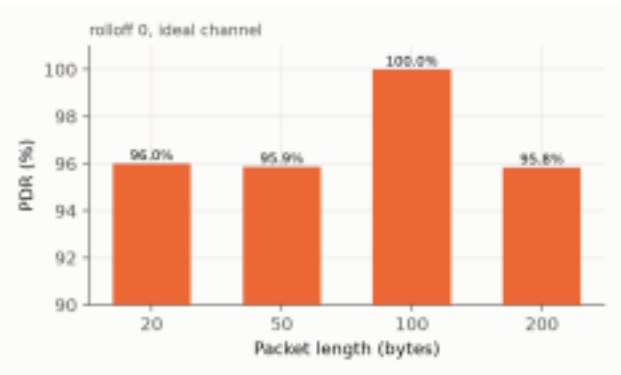
Figure 1. PDR per transmitter setting, 50-byte packets, no channel impairment. Blue: streaming, aqua: offline. Bars are means over 5 seeds × 500 packets; whiskers show min–max.

TX setting	Samples between bursts (measured)	Mean PDR (streaming)	Mean PDR (offline)	Lost packets per seed (both modes)
rolloff 0 (default)	0	95.87 %	95.88 %	26, 21, 19, 18, 19
rolloff 1	0 (GNU Radio resets it to 0 with a	95.88 %	95.88 %	26, 21, 19, 18, 19

	warning)			
rolloff 2	1 non-zero tail sample	99.84 %	99.84 %	0, 2, 2, 0, 0
rolloff 4	3 tapered tail samples	100 %	100 %	0, 0, 0, 0, 0
gap 1 (burst shaper)	1 zero sample	100 %	100 %	0, 0, 0, 0, 0
gap 8	8 zero samples	100 %	100 %	0, 0, 0, 0, 0

Interpretation. The lost-packet counts are the same in both modes for every seed. The scheduler moves data very differently in the two modes, so if it caused the drops, the counts would differ. This supports the explanation given by a contributor in the #6891 thread (§5.1) and rules out the scheduler hypothesis also raised there. Not every tail helps equally: one non zero tail sample (rolloff 2) reduces the loss, one zero sample removes it.

4.2 E2: packet length



20, 50 and 200-byte packets lose about 4 %; 100-byte packets lost none in all five seeds. We do not yet have an explanation for the 100-byte case. One hypothesis is that the burst length (in samples) changes how the next preamble's correlation window overlaps the previous burst's last symbol. This is a planned M3 measurement (logging the plateau-detector trigger positions). We report it as an open observation, not a finding.

Figure 2. PDR vs packet length, rolloff 0, ideal channel.

4.3 E3: behaviour with noise

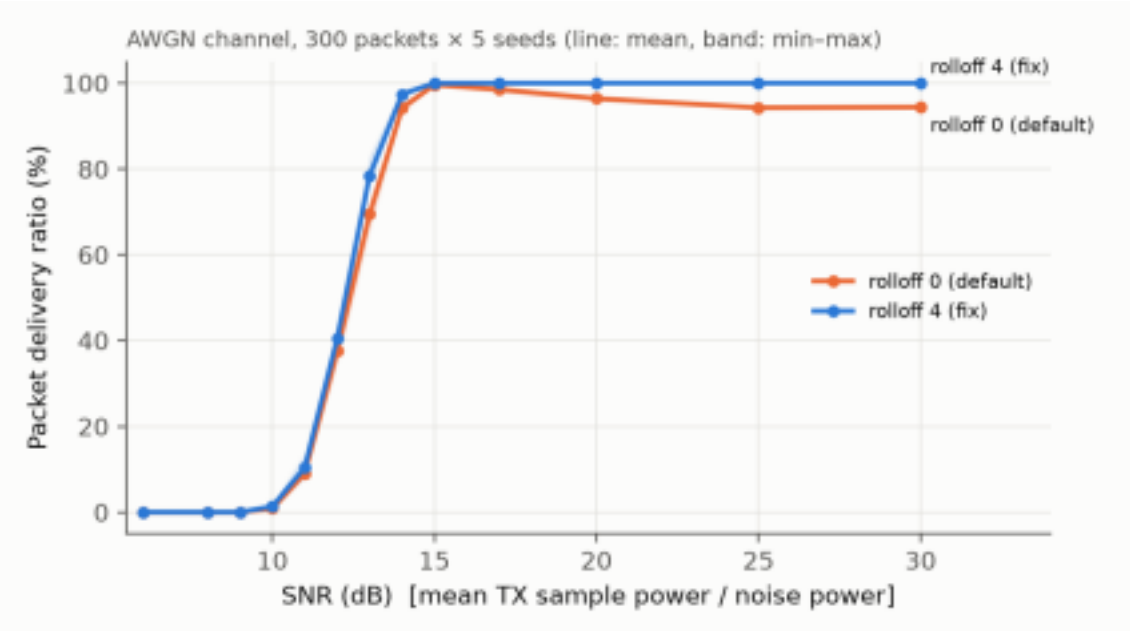


Figure 3. PDR vs SNR with an AWGN Channel Model. 300 packets × 5 seeds per point; band = min–max.

SNR (dB)	10	11	12	13	14	15	17	20	25	30
rolloff 0 PDR %	0.9	9.1	37.5	69.5	94.2	99.6	98.5	96.4	94.3	94.4
rolloff 4 PDR %	1.3	10.4	40.5	78.5	97.4	100	100	100	100	100

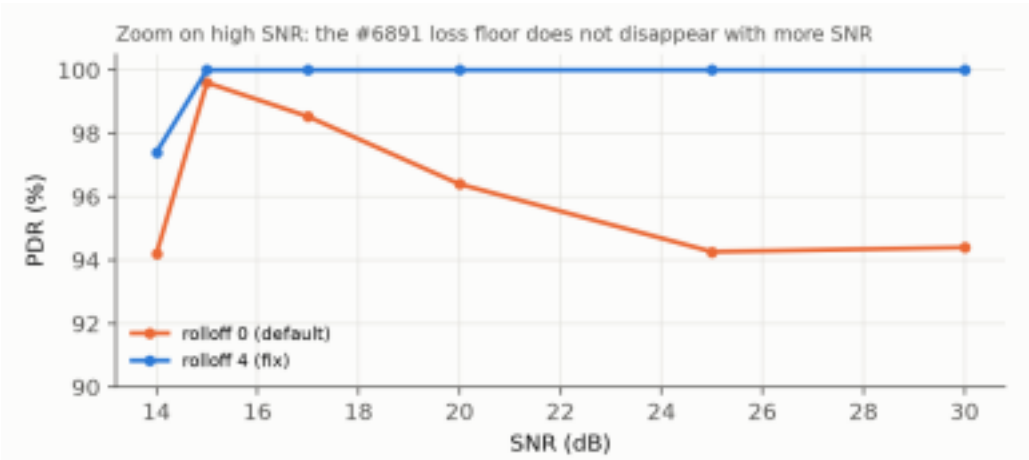


Figure 4. Zoom on SNR ≥ 14 dB: the default TX's loss returns as the channel gets cleaner.

Interpretation. Below 10 dB no packet survives: the BPSK header and CRC32 make the packet-level curve a steep "waterfall" between 10 and 15 dB. The fix never makes things worse and helps slightly even in the waterfall region. The default TX has a loss floor that grows with SNR. Our explanation, which still needs a direct measurement: noise randomises the shape of the Schmidl & Cox plateau, so detection timing is no longer biased early in the same way every time. A clean channel makes the error systematic. This matters for testing: a test run with some noise can pass even though the bug is still there.

4.4 E4 and E5: silent flowgraph freeze

Packet size (bytes)	**50**	**100**	**200**	**250**	**300**	**400**	**600**	**800**
Default buffers	OK	OK	OK	OK	HANG	HANG	HANG	HANG
Buffers 65 536 items	OK	OK	OK	OK	OK 20/20	OK 20/20	OK 20/20	OK 20/20
Block that failed					cyclic_prefixer	carrier_allocator	tagged_stream_mux	tagged_stream_m

E5 configuration (rolloff 4, 300 packets)	**Result**
No channel, RX length key = TX key (packet_len)	HANG: header_payload_demux "Buffer too small for min_noutput_items"
No channel, RX key rx_len	OK, 0.1 s
Channel Model block_tags=False , same key	HANG: same error
Channel Model block_tags=True , same key	OK, 0.1 s
Channel Model block_tags=False , key rx_len	OK, 0.1 s

In every HANG case, one block thread logs an error and stops. The remaining blocks wait for data that never arrives. Upstream blocks then stop because their output buffers are full (back pressure), so even the source stops. top_block.run() neither returns nor raises an exception. This is exactly the symptom reported in #7997: *"No blocks produce output anymore, including the source."*

5. Root-cause analysis

5.1 Packet loss (#6891)

1. Every frame starts with a preamble symbol whose two halves are identical. The Schmidl & Cox detector (ofdm_sync_sc_cfb) correlates the signal with itself shifted by half a symbol. Because of the cyclic prefix, the correlation has a *plateau* rather than a single peak, and a plateau detector picks a point on it as the frame start.

2. With back-to-back bursts, the correlation window around the next preamble also covers the end of the previous packet, whose content is random. So the plateau edge, and the chosen start point, moves by a few samples from packet to packet. A contributor to #6891 measured about ±3 samples.

3. Header/Payload Demux (header_payload_demux), once triggered, reads the header and then consumes the *whole* frame. Triggers that fall inside a frame it is already consuming are never seen.

4. Our packets are exactly 960 samples apart. If the next trigger comes even slightly early, it lands inside the current frame and that packet is lost. CRC32 hides this as a "missing packet", not as bit errors.

5. Separating bursts gives slack. We measured that one exact zero sample is enough, but one non-zero tail sample (rolloff 2) is not fully enough. The precise sample-level reason is still open; M3 will log trigger positions to confirm it.

Credit: the mechanism in steps 1–3 and the rolloff workaround were proposed in the #6891 discussion by GitHub user *ffu* (June 2026), building on observations by *Wiktorijus* and the confirmation by maintainer *marcusmueller* (Feb 2024) that this is "a rather serious" bug. Our contribution is the measurement that separates this from the scheduler hypothesis, the quantification, and the regression test.

5.2 Silent freeze

Tagged-stream blocks and Header/Payload Demux must output a complete packet or frame in one call. They request this with set_min_noutput_items() . In gr-digital/lib/ header_payload_demux_impl.cc (maint-3.10, function parse_header_data_msg), the demux sets this value from whatever length the header reports, and a comment in the source states that it cannot check whether the length is too large: *"There is currently no 'max payload len' param."* The runtime enforces the limit in gnuradio-runtime/lib/block_executor.cc (maint-3.10, lines 81– 83):

```
int best_n = round_down(out_buf->bufsize() / 2, output_multiple);
if (best_n < min_noutput_items)
 throw std::runtime_error("Buffer too small for min_noutput_items");
```

So a block may request at most half of its output buffer. The exception is caught by the per block thread wrapper, which logs it and ends that thread. Nothing stops the rest of the flowgraph, so it hangs. Raising the buffer with set_min_output_buffer() raises this limit, which is exactly what removed the freeze in E4.

A leaked TX length tag triggers the same path. The TX tag packet_len holds the burst length *in samples* (e.g. 963). If the RX uses the same key internally, a much larger length reaches the demux than the header actually specifies.

Status of #7997

The reporter's flowgraph (a custom Python pilot_inserter with general_work() / consume_each()) is not attached to the issue, so we cannot claim this is *their* root cause. We

have shown a mechanism that produces exactly their symptom with stock blocks. Inserting a Channel Model changes both tag propagation (block_tags) and buffer allocation, so it can plausibly push a working flowgraph over the edge. Our planned issue comment (§7) asks the reporter for the console log and the .grc so we can confirm or rule this out.

6. The contribution

6.1 What it changes

Two files in gnuradio/gnuradio (patches in Code/upstream_patch/):

gr-digital/python/digital/qa_ofdm_txrx.py (+29 lines)
+ _txrx_multipacket(n_packets, n_bytes, rolloff) helper: TX many packets, RX, return bytes
+ test_005_tx_multipacket_rolloff 100 back-to-back 50-byte packets, random payloads, rolloff=4, all bytes must match

gr-digital/examples/ofdm/ofdm_loopback.grc (1 line)
- rolloff: '0'
+ rolloff: '4'

6.2 Why it is technically meaningful

- It closes a real test gap. All five existing tests in qa_ofdm_txrx.py send a single packet. The failure in #6891 only happens *between* packets, so the test suite could not detect it. A regression here would go unnoticed.

- The test is shown to detect the bug. With rolloff=0 it fails (4850 != 5000 bytes: 3 packets lost); with rolloff=4 it passes. A test that cannot fail has no value, so we checked this directly.

- It fixes the official example. ofdm_loopback.grc is how most people first meet GNU Radio OFDM. With non-repeating data it drops about 4 % of packets, as the issue reporter found.

- It is small and low-risk. It follows the approach a contributor proposed in the issue thread, as the least-invasive change (no C++ change, no API change), so it is easy for maintainers to review.

- It is backed by data, not just "it works on my machine": 150 + 130 measured trials, with the choice of 4 justified by Fig. 1.

Contribution level (course guidelines §5.2): Level C (testing) with a Level A/B example fix. The proposed next contribution (§8) aims at Level B/D.

6.3 Testing done

$ python qa_ofdm_txrx.py # patched file, GNU Radio 3.10.12.0
Ran 6 tests in 0.156s
OK (3 repeated runs, all OK)

$ python qa_ofdm_txrx.py # same test with rolloff=0 (negative control) FAIL: test_005_tx_multipacket_rolloff
AssertionError: 4850 != 5000

6.4 Status

Step (course §5.3 workflow)	Status
Explore repository, read CONTRIBUTING.md and GREP1	Done
Select issue/task	Done (#6891 test gap; #7997 investigation)
Discuss with maintainers/community	Comments drafted (ISSUE_COMMENTS.md); to be posted
Implement and test	Done on GNU Radio 3.10.12 (conda)
Build GNU Radio main from source, run `ctest -R` qa_ofdm_txrx , open the example in GRC	Pending. GNU Radio's CONTRIBUTING.md asks contributors to test against their own build
Create pull request (signed-off commit, DCO)	Drafted (PR_DESCRIPTION.md , COMMIT_MESSAGE.txt); not yet opened
Respond to review → merge	Future

7. Maintainer and community response

Date	Who	What	What it means for us
Oct 2023	Reporter	Opens #6891 (drops in loopback example)	Problem is real for users
Feb 2024	marcusmueller (maintainer)	Reproduces it: "This is a bug, and it's a rather serious one"; rules out two hypotheses	Maintainers consider it important
Nov–Dec 2024	Wiktorijus	Isolates the S&C sync block; padding workaround	Points to signal processing
Jun 2026	ffu	Explains plateau + demux mechanism; recommends TX rolloff "if looking for the least change to the code base"	Our PR implements this recommendation

Jun 2026	Wiktorijus	Suspects buffers/scheduler	Our E1 (streaming = offline) answers this
Jan 2026	Arian-Taheri-12	Opens #7997 (freeze after inserting Channel Model)	No flowgraph attached
9 Oct 2026	vamsip-cell	Offers to investigate #7997	We coordinate instead of duplicating (comment drafted)

We have not posted yet, so we do not yet have a direct response to *our* work. We will post the drafted comments before opening the PR, as GNU Radio recommends discussing with maintainers first, and will record replies in the repository for M3.

8. Next planned contributions

1. Submit the PR (next 2 weeks): build GNU Radio main from source, run ctest , open the example in GRC, post the #6891 comment, open the PR, respond to review.

2. #7997 follow-up (M3): get the reporter's flowgraph; reproduce with a custom general_work() / consume_each() vector block; confirm or rule out the buffer/tag mechanism.

3. Fail loudly instead of freezing (Level B/D): propose that Header/Payload Demux reject a header whose length cannot fit its buffer (log a warning, drop that frame) instead of letting the thread die. The source comment shows maintainers already know this check is missing. We will propose it in an issue first, since it changes C++ behaviour.

4. Wi-Fi sensing bridge (later): ofdm_chanest_vcvc already computes per-packet channel estimates (CSI) but ofdm_rx does not expose them. A small example or option that outputs CSI per packet would let GNU Radio simulations with multipath Channel Models produce CSI datasets in the SenseFi format, so we can measure how lost packets affect sensing accuracy. This is our candidate GSoC-scale idea.

9. SOTA positioning in the open-source ecosystem

Project / work	**What it does**	**Relation to G1**
GNU Radio 3.10 gr-digital OFDM	Generic OFDM TX/RX (S&C sync, Header/Payload Demux, equalisers); used in examples and teaching	Our target. We measure and fix its packet-level reliability, which upstream tests do not cover

GNU Radio 4.0	New runtime with a different scheduler and buffer model	The freeze class depends on the 3.x thread-per-block runtime; our tests could be ported to check whether 4.0 avoids it
gr-ieee802-11 (Bloessl et al.)	Standard-compliant 802.11a/ g/p OFDM transceiver as a GNU Radio OOT module	Closest open SDR Wi-Fi PHY. Uses its own sync; our method (sequence numbered PDR, watchdog) can be applied to it
CSI tools: Linux 802.11n CSI Tool (Intel 5300), Atheros CSI Tool, Nexmon CSI, ESP32-CSI Tool, PicoScenes	Extract CSI from commercial Wi-Fi chips	Real hardware, but closed PHY and no control of the channel. Our direction is a *controlled, reproducible* simulated CSI source
SenseFi benchmark (Yang et al., 2023)	Datasets + deep models for CSI sensing	Application-level benchmark we will use to measure sensing impact (M3+)
IEEE 802.11bf (Du et al., 2024)	Standard for WLAN sensing procedures	Defines what sensing measurements should look like; motivates reliable per-packet CSI
GSoC 2026 GNU Radio projects: HIL CI (CortexLab), gr-cyberether, GR4 remote plotting	Hardware-in-the-loop testing, GPU visualisation, browser plotting	The HIL project already has an OFDM hardware test with PASS/FAIL analysis. Our multi-packet PDR test is a natural next HIL experiment

Where G1 stands

Existing Wi-Fi sensing work assumes a reliable stream of CSI samples and focuses on algorithms (survey papers 1–3, SenseFi). We work one layer below that assumption, on the reliability of the open-source OFDM receiver that would generate those samples. To our knowledge from the issue tracker and existing tests, no one has published a measurement that separates the scheduler and signal-processing explanations for #6891, quantified the SNR-dependent loss floor, or added a multi-packet regression test. We do not claim a new algorithm; we claim better-tested, better-understood infrastructure.

10. Limitations

- Simulation only; no SDR hardware yet. AWGN only; multipath and frequency offset are planned for M3.
- #7997's own flowgraph is unavailable (§5.2).

- Two observations are not yet explained: no loss at 100-byte packets (E2), and why one zero sample is enough but one non-zero tail sample is not (E1).

- Wi-Fi MTU-size packets (1500 B) also need the source-side buffer enlarged; not solved in M2.

- Tested on GNU Radio 3.10.12 binaries, not yet on a source build of main .

11. Reproducibility

Everything is in M2_g1_gsoc_wifisensing/ : Code/environment.yml (no administrator rights needed), four experiment scripts, Code/analysis/make_figures.py , raw CSV and logs in Results/ , and the patch in Code/upstream_patch/ . The README lists the exact commands. A full rerun takes about 10 minutes on a laptop.

12x. Use of generative AI

Following course rule §10: *[The team should state here honestly how AI tools were used, e.g. for explanation, code drafting and writing support, and confirm that every member has rerun the experiments, read the code and can explain every equation, script, figure and design decision in this report.]*

References

1. GNU Radio Issue #7997, "Complete OFDM Flowgraph Freeze After Inserting Channel Block in GNU Radio," 2026. https://github.com/gnuradio/gnuradio/issues/7997

2. GNU Radio Issue #6891, "OFDM Loopback Example drops packets even with 0 noise in simulation," 2023. https://github.com/gnuradio/gnuradio/issues/6891

3. T. M. Schmidl and D. C. Cox, "Robust frequency and timing synchronization for OFDM," *IEEE Trans. Communications*, vol. 45, no. 12, pp. 1613–1621, 1997.

4. GNU Radio source, gr-digital/lib/header_payload_demux_impl.cc , gr-digital/python/digital/ofdm_txrx.py , gr-digital/python/digital/qa_ofdm_txrx.py , CONTRIBUTING.md.

5. Y. Ma, G. Zhou, S. Wang, "WiFi sensing with channel state information: A survey," *ACM Computing Surveys*, 52(3), 2019.

6. R. Du et al., "An overview on IEEE 802.11bf: WLAN sensing," *IEEE Communications Surveys &*

Tutorials, 27(1), 2025.

7. A. Y. Radwan et al., "A Tutorial-Cum-Survey on Self-Supervised Learning for Wi-Fi Sensing," *IEEE COMST*, vol. 28, 2026.

8. J. Yang et al., "SenseFi: A Library and Benchmark on Deep-Learning-Empowered WiFi Human Sensing," *Patterns*, 4(3), 2023.

9. B. Bloessl et al., gr-ieee802-11: IEEE 802.11 a/g/p transceiver for GNU Radio. https://github.com/bastibl/gr ieee802-11

10. D. Halperin et al., "Tool release: Gathering 802.11n traces with channel state information," *ACM SIGCOMM CCR*, 41(1), 2011.

11. F. Gringoli et al., "Free your CSI: A channel state information extraction platform for modern Wi-Fi chipsets," *ACM WiNTECH*, 2019.

12. GSoC 2026 GNU Radio projects: S. Maity (HIL CI), Y. Mahmoud (gr-cyberether), Z. H. Fahmy (GR4 remote plotting). Repository links in Links_Papers.txt.